

TECHCATALYST DE 2026

dbt fundamentals

Turn SQL SELECT statements into tested, documented warehouse models with clear dependencies.

Week 5 Day 4

What dbt adds

Core, Fusion, extension

Two database boundary

Models, Jinja, ref()

Star schema and trust

Five questions to answer before the lab

- 1 What does dbt add beyond a SQL worksheet?
- 2 Where does AIRBNB.RAW end and your own schema begin?
- 3 How do Jinja, ref(), and source() become Snowflake SQL?
- 4 Why are some models views, tables, or incremental?
- 5 How do tests, snapshots, docs, and packages create trust?

dbt manages SQL transformations as a project

You provide

- SQL SELECT models
- Naming and business rules
- Dependencies through `ref()` and `source()`
- Tests and descriptions

dbt provides

- Compilation and run ordering
- Views, tables, and incremental builds
- DAG, documentation, and lineage
- Repeatable CLI commands

dbt is the T in ELT: data already lives in Snowflake, and dbt transforms it there.

Disconnected worksheets become difficult to trust

Without a managed project

- SQL copied between worksheets
- Physical names hardcoded by hand
- Run order remembered by people
- Data assumptions live in conversations

With dbt

- Models versioned and reviewed
- Logical references adapt by target
- Dependencies determine run order
- Tests and docs live beside the code

dbt does not replace Snowflake or ingestion. It organizes transformations on top of them.

Core runs today's lab; Fusion concepts still transfer

dbt Core v1

Python engine plus dbt-snowflake
The dbt CLI students run
Established path tested for class

Fusion CLI Preview

Newer engine written in Rust
Faster parsing and earlier feedback
Same dbt language and project shape

VS Code extension

Optional editor and lineage help
Ignore the registration screen
Right-click model, then dbt: View Lineage

Core executes the project. The extension helps inspect it. Fusion remains preview context.

Read from AIRBNB; build only inside TECHCATALYST

AIRBNB database

RAW schema, shared input

RAW_LISTINGS

RAW_REVIEWS

RAW_HOSTS

Read boundary

dbt Core project

student-work/week5/day4/airbnb

SQL, YAML, and Jinja

Compiles and runs transformations

Stores no business data itself

TECHCATALYST database

<STUDENTNAME> schema

Prefixed views and tables

src, dim, fct, mart

Snapshot history

AIRBNB.RAW → dbt Core → TECHCATALYST.<STUDENTNAME>

Project files have separate homes

```
airbnb/
├── dbt_project.yml # paths and defaults
├── models/
│   ├── src/      # staging views
│   ├── dim/      # dimension tables
│   ├── fct/      # incremental facts
│   └── mart/     # analysis-ready tables
├── macros/       # reusable Jinja
├── seeds/        # small CSV reference data
├── snapshots/    # row-history rules
└── tests/        # singular SQL tests

~/.dbt/profiles.yml # credentials and target
target/            # compiled SQL and metadata
```

Commit project code. Never commit profiles.yml, passwords, target/, or logs/.

Jinja renders a template before Snowflake sees SQL

Template model

```
SELECT *  
FROM {{ ref('src_listings') }}
```

SQL plus a Jinja expression

dbt compiles

ref() resolves the relation
Dependency edge is recorded
Project config chooses build
behavior

Snowflake executes

```
SELECT * FROM  
TECHCATALYST.JDOE.  
JDOE_SRC_LISTINGS
```

No curly braces remain

Template SQL + project context → compiled Snowflake SQL → warehouse object

source() marks the boundary; ref() connects models

```
-- Activity 3: raw input
WITH raw_listings AS (
  SELECT *
  FROM {{ source(
    'airbnb', 'listings'
  ) }}
)
```

```
-- Activity 2: dbt parent
WITH src_listings AS (
  SELECT *
  FROM {{ ref(
    'src_listings'
  ) }}
)
```

- source() points to dbt-unmanaged AIRBNB.RAW.
- ref() points to another dbt-built model.
- Both create lineage edges and allow dependency ordering.

One raw table becomes one staging view

Listings path

AIRBNB.RAW.RAW_LISTINGS



models/src/src_listings.sql



STUDENTNAME_SRC_LISTINGS view

Reviews path

AIRBNB.RAW.RAW_REVIEWS



models/src/src_reviews.sql



STUDENTNAME_SRC_REVIEWS view

Hosts path

AIRBNB.RAW.RAW_HOSTS



models/src/src_hosts.sql



STUDENTNAME_SRC_HOSTS view

No joins yet. Each staging model preserves one raw table's grain and standardizes names.

Use a CTE when it makes the query clearer

A CTE earns its place

- Multiple inputs or grains
- A reused intermediate result
- A step worth inspecting
- A complex join made readable

A direct SELECT is clearer

- One input relation
- One pass of renaming or casting
- No reused intermediate
- The shorter form stays readable

Activity 1 uses a CTE as a visible scaffold. Activity 2 genuinely needs CTEs for multi-input joins.

Facts store events; dimensions add context

Dimension: dim_listings_w_hosts

Grain: one row per listing

Describes room type and price

Includes host and superhost context

Key: listing_id

Fact: fct_reviews

Grain: one row per review

Records date, text, and sentiment

Grows as reviews arrive

Foreign key: listing_id

Join on listing_id. Always state the grain before joining two models.

A materialization decides what each SELECT becomes

view

- Stores query logic
- No stored result rows
- Fresh reads, repeated compute
- Used for src

table

- Stores result rows
- Full rebuild on dbt run
- Fast, predictable reads
- Used for dim and mart

incremental

- Table built fully once
- Later runs select new rows
- Needs explicit change rules
- Used for fct_reviews

The model defines the data. The materialization defines how dbt builds and stores it.

Two more built-ins complete the picture

ephemeral

Creates no warehouse object

Inlined as a CTE downstream

Useful for small helper logic

Can be harder to debug if overused

materialized_view

Warehouse-managed persisted view

Adapter support and behavior vary

Snowflake uses `dynamic_table` instead

Outside today's lab

dbt has five built-in materializations. Adapter support still determines what is available.

dbt_project.yml sets folder defaults

```
models:
  airbnb:
    +materialized: view
  src:
    +materialized: view
  dim:
    +materialized: table
  fct:
    +materialized: incremental
  mart:
    +materialized: table
```

Broad project default first, then folder overrides. A model-level config can override its folder.

Sources, seeds, and tests answer different questions

Source

Where does raw data begin?
Declares AIRBNB.RAW once
Referenced with source()
Owned outside dbt

Seed

Which small data belongs in Git?
Full moon date CSV
Loaded with dbt seed
Owned by the project

Test

What must be true?
Generic and singular checks
Zero invalid rows means pass
Run with dbt test or build

Activity 3 turns a working transformation graph into a trustworthy project.

Tests express the contract in YAML and SQL

```
models:
- name: dim_listings_cleansed
  columns:
  - name: listing_id
    data_tests:
    - unique
    - not_null
  - name: host_id
    data_tests:
    - relationships:
      arguments:
        to: ref('dim_hosts_cleansed')
        field: host_id
```

Generic tests reuse common checks. A singular test is a SELECT that returns invalid rows. Zero returned rows means pass.

History, documentation, and reusable logic come next

Snapshots

- Keep Type 2 row versions
- Interpret `dbt_valid_from` and `to`
- Controlled seed simulates change
- Run with `dbt snapshot`

Documentation

- Descriptions live with models
- Shows tests and compiled SQL
- DAG comes from `ref()` and `source()`
- Generate and serve locally

Packages

- Reuse reviewed project macros
- Pin `dbt_utils` 1.4.1
- Build a surrogate review key
- Run `dbt deps` first

Activity 4 is stretch work after `dbt build` is green.

Five activities, one connected Airbnb project

Goal: explain what changed, which dbt file caused it, and why the result is safer than a disconnected worksheet.

SELF-PACED

0. Core setup: install, connect, configure aliases and materializations.
1. First models: build three staging views and inspect compiled SQL.
2. Core models: dimensions, incremental fact, star schema, and lineage.
3. Trust layer: declare sources, load a seed, build a mart, and run nine tests.
4. Stretch: snapshots, generated docs, and dbt_utils package macros.

Deliverable: a green dbt build, correct objects in your schema, and an explanation you can defend.

If you can explain these five ideas, you are ready

- 1 dbt transforms data already in the warehouse. You write SELECT statements; dbt manages the build.
- 2 AIRBNB.RAW is shared input. TECHCATALYST.<STUDENTNAME> is your output boundary.
- 3 Jinja renders first. source() and ref() resolve names and form the dependency graph.
- 4 Grain separates facts from dimensions. Materialization controls how each model is stored.
- 5 Tests, snapshots, docs, and packages turn transformations into a maintainable project.